MODUL 04 · BAB II · ANALISIS RUNNING TIME

Insertion Sort: Cara Kerja dan Analisis Running Time

Algoritma pengurutan pertama kita, lengkap dengan penurunan running time kasus terbaik (linier) dan terburuk (kuadrat).

TUJUAN PEMBELAJARAN

Setelah mempelajari modul ini, kamu diharapkan mampu:

- Menjelaskan gagasan Insertion Sort dan arti setiap baris pseudocode-nya.
- Menelusuri Insertion Sort pada array kecil dan mencatat keadaan array setiap putaran.
- Menyusun tabel cost × jumlah operasi dengan variabel t_j.
- Menurunkan running time kasus terbaik $T(n) = an + b$ dan kasus terburuk $T(n) = an^2 + bn + c$.
- Menentukan jenis masukan yang menghasilkan kasus terbaik dan terburuk.

PRASYARAT

- Modul 03: metode cost × jumlah operasi, kasus terbaik dan terburuk, deret aritmetika.

BAB / MATERI ASAL

Materi 2. ALGORITMA INSERTION SORT dan RUNNING TIME

POSISI DALAM URUTAN BELAJAR

Penerapan khusus Modul 03. Hasilnya (linier vs kuadrat) dipakai sebagai contoh di Modul 05 dan 06, lalu dibandingkan dengan Merge Sort di Modul 08.

DAFTAR ISI

1. Mengurutkan Seperti Menyusun Kartu 1
2. Algoritma Insertion Sort 2
3. Penelusuran: Mengurutkan 12, 10, 15, 7 3
4. Analisis Running Time 4
5. Makna Hasil Analisis 7
6. Kesimpulan 8
- Kuis Pemahaman 9
- Kunci Jawaban dan Pembahasan 10
- Referensi 11

PENDAHULUAN

1. Mengurutkan Seperti Menyusun Kartu

Bayangkan kamu bermain kartu. Kartu dibagikan satu per satu, dan setiap kali menerima kartu baru, kamu menyelipkannya ke posisi yang tepat di antara kartu-kartu yang sudah kamu pegang. Kartu di tanganmu selalu dalam keadaan terurut. Setelah kartu terakhir diselipkan, seluruh kartu terurut.

Itulah cara kerja **Insertion Sort** (pengurutan penyisipan). Algoritma ini sederhana, mudah ditelusuri dengan tangan, dan sangat cocok sebagai contoh pertama untuk mempraktikkan analisis running time dari Modul 03. Insertion Sort juga menarik karena running time-nya sangat bergantung pada isi data: untuk data yang

sudah terurut ia bekerja dalam waktu linier, sedangkan untuk data yang terurut terbalik ia membutuhkan waktu kuadrat.

Modul ini membahas algoritmanya baris demi baris, menelusurinya dengan contoh dari bahan kuliah, lalu menurunkan rumus running time untuk kasus terbaik dan terburuk secara lengkap.

PEMBAHASAN INTI

2. Algoritma Insertion Sort

2.1 Masalah pengurutan

Masukan: array $A[1..n]$ berisi n bilangan sembarang. **Keluaran:** array yang sama, isinya disusun ulang sehingga $A[1] \leq A[2] \leq \cdots \leq A[n]$ (terurut dari kecil ke besar). Insertion Sort mengurutkan **di tempat** (*in place*): elemen-elemen disusun ulang di dalam array A itu sendiri tanpa array bantu.

Sebagai pengingat notasi array dari bahan kuliah: jika barisan 10, 20, 30, 40 disimpan di array A berukuran 1×4, maka $A[1] = 10$, $A[2] = 20$, $A[3] = 30$, $A[4] = 40$. Bilangan 1, 2, 3, 4 disebut **indeks**, sedangkan 10, 20, 30, 40 adalah **isi** array.

2.2 Pseudocode

INSERTION_SORT(A) // A: array satu dimensi berisi n bilangan

```
1  for j = 2 to length[A]
2      key = A[j]
3      i = j - 1
4      while i > 0 and A[i] > key
5          A[i + 1] = A[i]
6          i = i - 1
7      A[i + 1] = key
```

2.3 Penjelasan setiap baris

Baris	Penjelasan
1	Perulangan `for` untuk $j = 2$ sampai panjang array. Elemen $A[j]$ adalah "kartu baru" yang akan disisipkan. Ketika j = panjang array + 1, perulangan berakhir dan algoritma selesai. Perulangan dimulai dari 2 karena $A[1]$ saja sudah dianggap terurut.
2	Simpan nilai $A[j]$ ke variabel `key`. Penyimpanan ini perlu karena posisi $A[j]$ nanti dapat tertimpa oleh pergeseran.
3	Mulai membandingkan dari elemen tepat di sebelah kiri `key`, yaitu indeks $i = j - 1$.
4	Ulangi selama $i > 0$ (belum melewati ujung kiri array) **dan** $A[i] > key$ (elemen di kiri masih lebih besar). Jika kondisi benar, jalankan baris 5 dan 6; jika salah, keluar dari `while` ke baris 7.
5	Geser $A[i]$ satu posisi ke kanan, ke $A[i + 1]$.
6	Mundur satu posisi: $i = i - 1$.
7	Letakkan `key` di posisi kosong yang ditemukan, yaitu $A[i + 1]$.

Tabel 1. Arti setiap langkah INSERTION_SORT.

MENGAPA ALGORITMA INI BENAR?

Pada awal setiap putaran `for` dengan nilai j, subarray $A[1..j-1]$ berisi elemen-elemen asli posisi 1 sampai $j-1$, **sudah dalam keadaan terurut**. Sifat yang selalu benar di awal setiap putaran ini disebut **invarian perulangan** (*loop invariant*). Sifat ini benar di awal ($j=2$, subarray $A[1..1]$ hanya berisi satu elemen), tetap benar setelah setiap putaran (karena `key` disisipkan di posisi yang tepat), dan ketika perulangan berakhir dengan $j=n+1$, sifat itu menyatakan bahwa $A[1..n]$ terurut. Cara pembuktian ini diuraikan lengkap oleh Cormen dkk. (2009, Bab 2.1).

CONTOH DAN PENERAPAN

3. Penelusuran: Mengurutkan 12, 10, 15, 7

Contoh dari bahan kuliah. Bilangan 12, 10, 15, 7 disimpan di array A berukuran 1×4. Sel kuning menandai `key` yang sedang disisipkan; sel biru menandai posisi tempat `key` diletakkan.

indeks	1	2	3	4
A	12	10	15	7

Tabel 2. Keadaan awal.

3.1 Loop 1 (j = 2)

```
key = A[2] = 10,  i = 1
while 1 > 0 and A[1] > key  →  TRUE and (12 > 10) TRUE  →  TRUE
    A[2] = A[1] = 12          → A = [12, 12, 15, 7]
    i = 0
while 0 > 0 ...  →  FALSE (end while)
A[1] = key = 10               → A = [10, 12, 15, 7]
```

3.2 Loop 2 (j = 3)

```
key = A[3] = 15,  i = 2
while 2 > 0 and A[2] > key  →  TRUE and (12 > 15) FALSE  →  FALSE (end while)
A[3] = key = 15               → A = [10, 12, 15, 7]   (tidak ada pergeseran)
```

3.3 Loop 3 (j = 4)

```
key = A[4] = 7,  i = 3
while 3 > 0 and A[3] > key  →  (15 > 7) TRUE   →  A[4] = 15, i = 2   → A = [10, 12, 15, 15]
while 2 > 0 and A[2] > key  →  (12 > 7) TRUE   →  A[3] = 12, i = 1   → A = [10, 12, 12, 15]
while 1 > 0 and A[1] > key  →  (10 > 7) TRUE   →  A[2] = 10, i = 0   → A = [10, 10, 12, 15]
while 0 > 0 ...             →  FALSE (end while)
A[1] = key = 7                → A = [7, 10, 12, 15]
```

3.4 Loop 4 (j = 5)

$j=5$ melebihi panjang array (4), sehingga perulangan `for` berakhir. Algoritma selesai dengan hasil:

indeks	1	2	3	4
A	7	10	12	15

Tabel 3. Keadaan akhir: terurut.

j	key	Pergeseran	t_j (pemeriksaan while)	Array setelah putaran
2	10	1	2	10, 12, 15, 7
3	15	0	1	10, 12, 15, 7
4	7	3	4	7, 10, 12, 15

Tabel 4. Ringkasan: banyaknya pemeriksaan kondisi while (t_j) pada setiap putaran.

Perhatikan pola pentingnya: t_j **selalu sama dengan banyaknya pergeseran ditambah satu**. Satu pemeriksaan tambahan adalah pemeriksaan terakhir yang bernilai salah. Pola ini menjadi kunci analisis berikutnya.

PEMBAHASAN INTI

4. Analisis Running Time

4.1 Tabel cost dan jumlah operasi

Kita definisikan t_j = banyaknya kali kondisi `while` pada baris 4 diperiksa untuk nilai j tertentu. Nilai t_j bergantung pada isi array, sehingga untuk sementara kita biarkan sebagai variabel.

Baris	Pernyataan	Cost	Jumlah operasi
1	`for j = 2 to length[A]`	c_1	n
2	`key = A[j]`	c_2	$n-1$
3	`i = j - 1`	c_3	$n-1$
4	`while i > 0 and A[i] > key`	c_4	$\sum_{j=2}^{n} t_j$
5	`A[i+1] = A[i]`	c_5	$\sum_{j=2}^{n}(t_j-1)$
6	`i = i - 1`	c_6	$\sum_{j=2}^{n}(t_j-1)$
7	`A[i+1] = key`	c_7	$n-1$

Tabel 5. Cost dan jumlah operasi setiap baris INSERTION_SORT.

Alasan setiap jumlah operasi:

- **Baris 1** dijalankan n kali: untuk $j = 2, \ldots, n$ (yaitu $n-1$ kali) ditambah satu kali saat $j = n+1$ yang mengakhiri perulangan.
- **Baris 2, 3, 7** berada di dalam `for` tetapi di luar `while`, sehingga dijalankan sekali untuk setiap $j = 2, \ldots, n$, yaitu $n-1$ kali.
- **Baris 4** dijalankan t_j kali untuk setiap j, sehingga totalnya $t_2 + t_3 + \cdots + t_n = \sum_{j=2}^{n} t_j$.
- **Baris 5 dan 6** adalah isi `while`, yang dijalankan satu kali lebih sedikit dari kondisinya (pemeriksaan terakhir bernilai salah), yaitu $t_j - 1$ kali untuk setiap j.

Running time adalah jumlah dari cost dikali jumlah operasi:

$$T(n) = c_1 n + c_2(n-1) + c_3(n-1) + c_4 \sum_{j=2}^{n} t_j + c_5 \sum_{j=2}^{n} (t_j - 1) + c_6 \sum_{j=2}^{n} (t_j - 1) + c_7(n-1).$$

4.2 Kasus terbaik: array sudah terurut

Jika array sudah terurut dari kecil ke besar, untuk setiap j kondisi $A[i] > key$ langsung bernilai salah pada pemeriksaan pertama. Tidak ada pergeseran, sehingga $t_j = 1$ **untuk setiap** j. Contoh dari bahan kuliah, $A = (2, 4, 12, 15, 30)$:

Penelusuran kasus terbaik A = (2, 4, 12, 15, 30)

```
Loop 1: j = 2, key = 4,  i = 1   while: A[1] = 2  > 4 ?  FALSE   A[2] = 4
Loop 2: j = 3, key = 12, i = 2   while: A[2] = 4  > 12?  FALSE   A[3] = 12
Loop 3: j = 4, key = 15, i = 3   while: A[3] = 12 > 15?  FALSE   A[4] = 15
Loop 4: j = 5, key = 30, i = 4   while: A[4] = 15 > 30?  FALSE   A[5] = 30
Loop 5: j = 6  → end for, selesai.     (setiap t_j = 1)
```

Dengan $t_j = 1$:

$$\sum_{j=2}^{n} t_j = \sum_{j=2}^{n} 1 = n - 1, \qquad \sum_{j=2}^{n} (t_j - 1) = 0.$$

Perhatikan bahwa penjumlahan dimulai dari $j = 2$, sehingga ada $n - 1$ suku bernilai 1. Substitusikan ke rumus $T(n)$:

$$\begin{aligned} T(n) &= c_1 n + c_2(n-1) + c_3(n-1) + c_4(n-1) + c_5 \cdot 0 + c_6 \cdot 0 + c_7(n-1) \\ &= (c_1 + c_2 + c_3 + c_4 + c_7)\, n - (c_2 + c_3 + c_4 + c_7). \end{aligned}$$

Dengan $a = c_1 + c_2 + c_3 + c_4 + c_7$ dan $b = -(c_2 + c_3 + c_4 + c_7)$, diperoleh

HASIL KASUS TERBAIK

$$T(n) = an + b \quad (\text{fungsi linier}).$$

4.3 Kasus terburuk: array terurut terbalik

Jika array terurut dari besar ke kecil, setiap `key` lebih kecil dari semua elemen di kirinya. Semua $j - 1$ elemen di kiri harus digeser, lalu kondisi diperiksa sekali lagi ketika $i = 0$. Jadi $t_j = (j - 1) + 1 = j$. Contoh dari bahan kuliah, $A = (30, 15, 12, 4, 2)$:

j	key	Pergeseran	t_j	Array setelah putaran
2	15	1	2	15, 30, 12, 4, 2
3	12	2	3	12, 15, 30, 4, 2
4	4	3	4	4, 12, 15, 30, 2
5	2	4	5	2, 4, 12, 15, 30

Tabel 6. Kasus terburuk A = (30, 15, 12, 4, 2): setiap key bergeser sampai ujung kiri.

Dengan $t_j = j$, kita butuh deret aritmetika $1 + 2 + \cdots + n = \frac{n(n+1)}{2}$. Karena penjumlahan dimulai dari $j = 2$, suku $j = 1$ dikurangkan:

$$\sum_{j=2}^{n} t_j = \sum_{j=2}^{n} j = \frac{n(n+1)}{2} - 1 = \frac{n^2 + n - 2}{2},$$

$$\sum_{j=2}^{n} (t_j - 1) = \sum_{j=2}^{n} j - \sum_{j=2}^{n} 1 = \frac{n^2 + n - 2}{2} - (n-1) = \frac{n^2 - n}{2}.$$

Substitusikan ke rumus $T(n)$:

$$\begin{aligned} T(n) &= c_1 n + c_2(n-1) + c_3(n-1) + c_4\left(\frac{n^2+n-2}{2}\right) + c_5\left(\frac{n^2-n}{2}\right) + c_6\left(\frac{n^2-n}{2}\right) + c_7(n-1) \\ &= \left(\frac{c_4 + c_5 + c_6}{2}\right) n^2 + \left(c_1 + c_2 + c_3 + \frac{c_4 - c_5 - c_6}{2} + c_7\right) n - (c_2 + c_3 + c_4 + c_7). \end{aligned}$$

Dengan $a = \frac{c_4+c_5+c_6}{2}$, $b = c_1 + c_2 + c_3 + \frac{c_4-c_5-c_6}{2} + c_7$, dan $c = -(c_2 + c_3 + c_4 + c_7)$:

HASIL KASUS TERBURUK

$$T(n) = an^2 + bn + c \quad (\text{fungsi kuadrat}).$$

4.4 Kasus rata-rata

Bahan kuliah hanya membahas kasus terbaik dan terburuk. Sebagai pelengkap: jika urutan awal data acak, rata-rata setiap `key` perlu dibandingkan dengan sekitar separuh elemen di kirinya, sehingga $t_j \approx j/2$. Penjumlahannya tetap menghasilkan suku n^2 (dengan koefisien kira-kira separuh kasus terburuk), sehingga kasus rata-rata Insertion Sort juga berorde **kuadrat** (Cormen dkk., 2009, Bab 2.2).

4.5 Verifikasi dengan program

Program berikut menjalankan Insertion Sort sambil menghitung $\sum t_j$ (eksekusi baris 4) dan $\sum (t_j - 1)$ (eksekusi baris 5). Hasilnya dibandingkan dengan rumus kasus terbaik ($n - 1$ dan 0) serta kasus terburuk ($\frac{n^2+n-2}{2}$ dan $\frac{n^2-n}{2}$).

Python: Insertion Sort dengan penghitung t_j

```python
def insertion_sort(arr):
    A = [None] + list(arr)          # A[1..n] seperti pseudocode
    n = len(arr)
    sum_t, sum_geser = 0, 0
    for j in range(2, n + 1):
        key = A[j]
        i = j - 1
        t = 1                       # pemeriksaan while yang pertama
        while i > 0 and A[i] > key:
            A[i + 1] = A[i]
            i = i - 1
            sum_geser += 1
            t += 1                  # pemeriksaan while berikutnya
        A[i + 1] = key
        sum_t += t
    return A[1:], sum_t, sum_geser

for data in ([12, 10, 15, 7], [2, 4, 12, 15, 30], [30, 15, 12, 4, 2]):
    print(data, "->", insertion_sort(data))

n = 1000
_, t_best, g_best = insertion_sort(range(n))            # terurut
_, t_worst, g_worst = insertion_sort(range(n, 0, -1))   # terurut terbalik
print(t_best, g_best, n - 1)
print(t_worst, g_worst, (n*n + n - 2)//2, (n*n - n)//2)
```

Keluaran program

```
[12, 10, 15, 7] -> ([7, 10, 12, 15], 7, 4)
[2, 4, 12, 15, 30] -> ([2, 4, 12, 15, 30], 4, 0)
[30, 15, 12, 4, 2] -> ([2, 4, 12, 15, 30], 14, 10)
999 0 999
500499 499500 500499 499500
```

Untuk contoh 12, 10, 15, 7: $\sum t_j = 2 + 1 + 4 = 7$ dan pergeseran $= 1 + 0 + 3 = 4$, sesuai penelusuran manual. Untuk $n = 1000$, hitungan program sama persis dengan rumus.

ANALISIS DAN HUBUNGAN KONSEP

5. Makna Hasil Analisis

Isi data menentukan bentuk running time. Insertion Sort adalah contoh algoritma yang kasus terbaik dan terburuknya memiliki orde berbeda: linier untuk data terurut, kuadrat untuk data terurut terbalik. Dengan $n = 1000$, banyaknya pergeseran berkisar dari 0 sampai 499.500. Dalam notasi Modul 06, kasus terbaiknya $\Theta(n)$ dan kasus terburuknya $\Theta(n^2)$.

Banyaknya pergeseran = banyaknya inversi. Setiap pergeseran memperbaiki tepat satu pasangan elemen yang urutannya terbalik (disebut **inversi**). Karena itu Insertion Sort sangat cepat untuk data yang "hampir terurut" (inversinya sedikit) dan lambat untuk data yang banyak inversinya.

Kapan Insertion Sort dipakai? Ia cocok untuk data berukuran kecil atau data yang hampir terurut, mengurutkan di tempat (memori tambahan konstan), dan bersifat **stabil**: elemen bernilai sama tidak

bertukar urutan karena kondisi pergeseran memakai $>$ dan bukan $\geq$. Untuk data besar yang acak, orde kuadratnya menjadi masalah serius. Modul 05 menunjukkan seberapa cepat n^2 tumbuh dibanding $n \lg n$, dan Modul 08 memperkenalkan Merge Sort yang kasus terburuknya $\Theta(n \lg n)$.

Sifat	Nilai
Kasus terbaik (data terurut)	$T(n) = an + b$, linier
Kasus terburuk (data terurut terbalik)	$T(n) = an^2 + bn + c$, kuadrat
Kasus rata-rata (data acak)	Kuadrat
Memori tambahan	Konstan (hanya `key`, `i`, `j`)
Stabil?	Ya

Tabel 7. Ringkasan sifat Insertion Sort.

KESIMPULAN

6. Kesimpulan

- Insertion Sort menyisipkan $A[j]$ ke posisi yang tepat di subarray $A[1..j-1]$ yang sudah terurut, untuk $j = 2, \ldots, n$.
- Variabel t_j menyatakan banyaknya pemeriksaan kondisi `while` untuk setiap j, sama dengan banyaknya pergeseran ditambah satu.
- Running time umum: $T(n) = c_1 n + (c_2 + c_3 + c_7)(n-1) + c_4 \sum t_j + (c_5 + c_6) \sum (t_j - 1)$.
- Kasus terbaik (data terurut): $t_j = 1$, sehingga $T(n) = an + b$ (linier).
- Kasus terburuk (data terurut terbalik): $t_j = j$, sehingga $T(n) = an^2 + bn + c$ (kuadrat). Kasus rata-rata juga kuadrat.

Kuis Pemahaman

Kerjakan semua soal terlebih dahulu tanpa melihat kunci jawaban. Kunci jawaban beserta pembahasannya ada di halaman berikutnya.

1 PENELUSURAN

Urutkan $A = (8, 3, 5, 1)$ dengan Insertion Sort. Tuliskan keadaan array setelah setiap putaran j dan nilai t_j untuk setiap j.

2 PILIHAN GANDA

Mengapa nilai $A[j]$ perlu disimpan di variabel `key` pada baris 2?

A. Agar perulangan `for` dapat berhenti.
B. Karena posisi $A[j]$ dapat tertimpa oleh pergeseran pada baris 5.
C. Karena `key` dipakai untuk menghitung t_j.
D. Agar array dapat diurutkan dari besar ke kecil.

3 PERHITUNGAN

Untuk $n = 6$, berapa total eksekusi baris 4 ($\sum t_j$) dan baris 5 ($\sum (t_j - 1)$) pada (a) kasus terbaik dan (b) kasus terburuk?

4 BENAR/SALAH DENGAN ALASAN

"Baris 1 (`for`) Insertion Sort dijalankan $n - 1$ kali karena j bergerak dari 2 sampai n." Benar atau salah?

5 ANALISIS

Array $A = (1, 2, 3, 4, 5, 6, 7, 0)$ hampir terurut: hanya elemen terakhir yang salah tempat. Berapa $\sum (t_j - 1)$ (banyaknya pergeseran)? Apakah ini kasus terbaik, terburuk, atau di antaranya?

6 PILIHAN GANDA

Jika kondisi pada baris 4 diubah dari `A[i] > key` menjadi `A[i] ≥ key`, apa akibatnya?

A. Hasil pengurutan menjadi salah.
B. Array tetap terurut, tetapi elemen yang bernilai sama bisa bertukar urutan (tidak stabil) dan pergeseran bisa bertambah.
C. Algoritma tidak pernah berhenti.
D. Running time kasus terburuk menjadi linier.

7 ANALISIS

Jelaskan mengapa $t_j = j$ pada kasus terburuk, bukan $t_j = j - 1$.

8 PERHITUNGAN

Sebuah program Insertion Sort membutuhkan 0,01 detik untuk mengurutkan 2.000 data yang terurut terbalik. Perkirakan waktunya untuk 20.000 data yang terurut terbalik, dengan asumsi suku n^2 dominan.

Kunci Jawaban dan Pembahasan

Bandingkan jawabanmu, lalu baca pembahasannya untuk memahami alasan di balik setiap jawaban.

Soal 1

j	key	Array setelah putaran	t_j
2	3	3, 8, 5, 1	2
3	5	3, 5, 8, 1	2
4	1	1, 3, 5, 8	4

Pada $j = 3$, key 5 menggeser 8 (satu pergeseran) lalu berhenti karena $3 > 5$ salah, jadi $t_3 = 2$. Pada $j = 4$, key 1 menggeser 8, 5, 3 lalu berhenti karena $i = 0$, jadi $t_4 = 4$.

Soal 2

Jawaban: B. Pada pergeseran pertama, baris 5 menulis $A[j] = A[j-1]$, sehingga nilai asli $A[j]$ hilang bila tidak disimpan lebih dulu.

Soal 3

(a) Kasus terbaik: $\sum t_j = n - 1 = 5$, $\sum (t_j - 1) = 0$. (b) Kasus terburuk: $\sum t_j = \frac{36+6-2}{2} = 20$ dan $\sum (t_j - 1) = \frac{36-6}{2} = 15$. Cek: $t_j = 2, 3, 4, 5, 6$ berjumlah 20, dan pergeserannya $1 + 2 + 3 + 4 + 5 = 15$.

Soal 4

Salah. Isi perulangan memang dijalankan $n - 1$ kali, tetapi kepala perulangan dijalankan n **kali**, termasuk pemeriksaan $j = n + 1$ yang mengakhiri perulangan.

Soal 5

Untuk $j = 2, \ldots, 7$ tidak ada pergeseran. Untuk $j = 8$, key 0 harus melewati ketujuh elemen, sehingga ada 7 pergeseran. Total $\sum (t_j - 1) = 7$. Ini **di antara** kasus terbaik (0) dan terburuk ($\frac{64-8}{2} = 28$), dan jauh lebih dekat ke kasus terbaik. Banyaknya pergeseran sama dengan banyaknya inversi, yaitu 7 pasangan $(k, 0)$.

Soal 6

Jawaban: B. Dengan $\geq$, elemen yang nilainya sama dengan key juga ikut digeser, sehingga key ditempatkan **sebelum** elemen bernilai sama yang semula di kirinya. Hasilnya tetap terurut tetapi tidak stabil, dan pergeseran bertambah bila ada nilai yang sama. Algoritma tetap berhenti karena i terus berkurang.

Soal 7

Pada kasus terburuk key harus melewati semua $j - 1$ elemen di kirinya, sehingga kondisi `while` bernilai benar $j - 1$ kali. Setelah itu $i = 0$ dan kondisi diperiksa **sekali lagi** dengan hasil salah. Total pemeriksaan $= (j - 1) + 1 = j$.

Soal 8

Ukuran data naik $\frac{20000}{2000} = 10$ kali. Karena suku n^2 dominan, waktunya naik sekitar $10^2 = 100$ kali, yaitu kira-kira $0{,}01 \times 100 = 1$ **detik**.

Referensi

Sumber utama: Materi 2. ALGORITMA INSERTION SORT dan RUNNING TIME (bahan kuliah Perancangan dan Analisis Algoritma). Penjelasan tambahan, koreksi, dan pengayaan mengacu pada pustaka berikut.

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). Cambridge, MA: MIT Press. ISBN 978-0-262-03384-8.
2. Knuth, D. E. (1997). *The Art of Computer Programming, Vol. 1: Fundamental Algorithms* (3rd ed.). Reading, MA: Addison-Wesley. (Bagian 1.1 tentang lima sifat algoritma.)
3. Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Upper Saddle River, NJ: Addison-Wesley.